In [1]:
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.linear_model import LogisticRegression
```

In [2]:
```python
df = pd.read_csv("brca.csv")

df.head()
```

Out[2]:

	Unnamed: 0	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean
0	1	13.540	14.36	87.46	566.3
1	2	13.080	15.71	85.63	520.0
2	3	9.504	12.44	60.34	273.9
3	4	13.030	18.42	82.61	523.8
4	5	8.196	16.84	51.71	201.9

5 rows × 32 columns

In [3]:
```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   Unnamed: 0            569 non-null    int64  
 1   x.radius_mean         569 non-null    float64
 2   x.texture_mean        569 non-null    float64
 3   x.perimeter_mean      569 non-null    float64
 4   x.area_mean           569 non-null    float64
 5   x.smoothness_mean     569 non-null    float64
 6   x.compactness_mean    569 non-null    float64
 7   x.concavity_mean      569 non-null    float64
 8   x.concave_pts_mean    569 non-null    float64
 9   x.symmetry_mean       569 non-null    float64
 10  x.fractal_dim_mean    569 non-null    float64
 11  x.radius_se           569 non-null    float64
 12  x.texture_se          569 non-null    float64
 13  x.perimeter_se        569 non-null    float64
 14  x.area_se             569 non-null    float64
 15  x.smoothness_se       569 non-null    float64
 16  x.compactness_se      569 non-null    float64
 17  x.concavity_se        569 non-null    float64
 18  x.concave_pts_se      569 non-null    float64
 19  x.symmetry_se         569 non-null    float64
 20  x.fractal_dim_se      569 non-null    float64
 21  x.radius_worst        569 non-null    float64
 22  x.texture_worst       569 non-null    float64
 23  x.perimeter_worst     569 non-null    float64
 24  x.area_worst          569 non-null    float64
 25  x.smoothness_worst    569 non-null    float64
 26  x.compactness_worst   569 non-null    float64
 27  x.concavity_worst     569 non-null    float64
 28  x.concave_pts_worst   569 non-null    float64
 29  x.symmetry_worst      569 non-null    float64
 30  x.fractal_dim_worst   569 non-null    float64
 31  y                     569 non-null    object 
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB
```

In [4]:
```
df.isnull().sum()
```

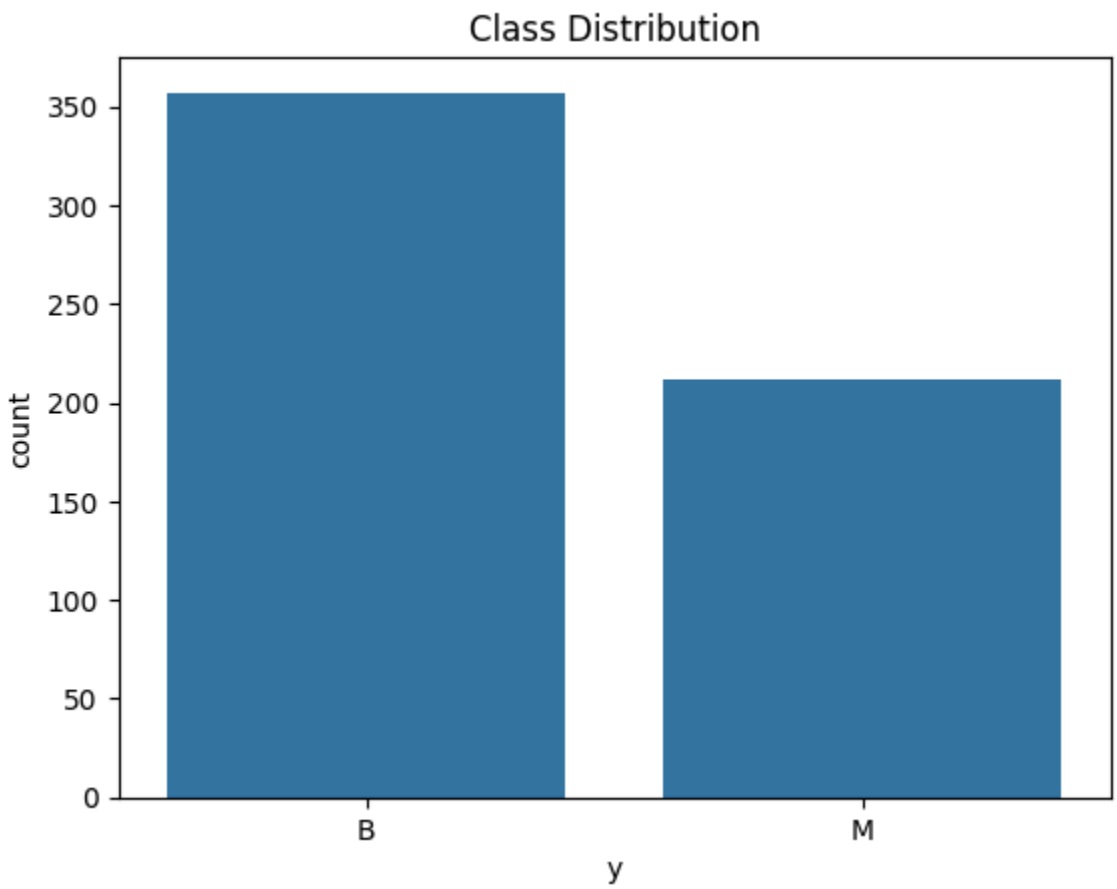
Out[4]:

	0
Unnamed: 0	0
x.radius_mean	0
x.texture_mean	0
x.perimeter_mean	0
x.area_mean	0
x.smoothness_mean	0
x.compactness_mean	0
x.concavity_mean	0
x.concave_pts_mean	0
x.symmetry_mean	0
x.fractal_dim_mean	0
x.radius_se	0
x.texture_se	0
x.perimeter_se	0
x.area_se	0
x.smoothness_se	0
x.compactness_se	0
x.concavity_se	0
x.concave_pts_se	0
x.symmetry_se	0
x.fractal_dim_se	0
x.radius_worst	0
x.texture_worst	0
x.perimeter_worst	0
x.area_worst	0
x.smoothness_worst	0
x.compactness_worst	0
x.concavity_worst	0
x.concave_pts_worst	0
x.symmetry_worst	0
x.fractal_dim_worst	0

	0
y	0

dtype: int64

In [5]:
```
sns.countplot(x='y', data=df)
plt.title("Class Distribution")
plt.show()
```



In [6]:
```
df = df.drop(columns=["Unnamed: 0"])
```

In [7]:
```
df['y'] = df['y'].map({'B':1, 'M':-1})

df.head()
```

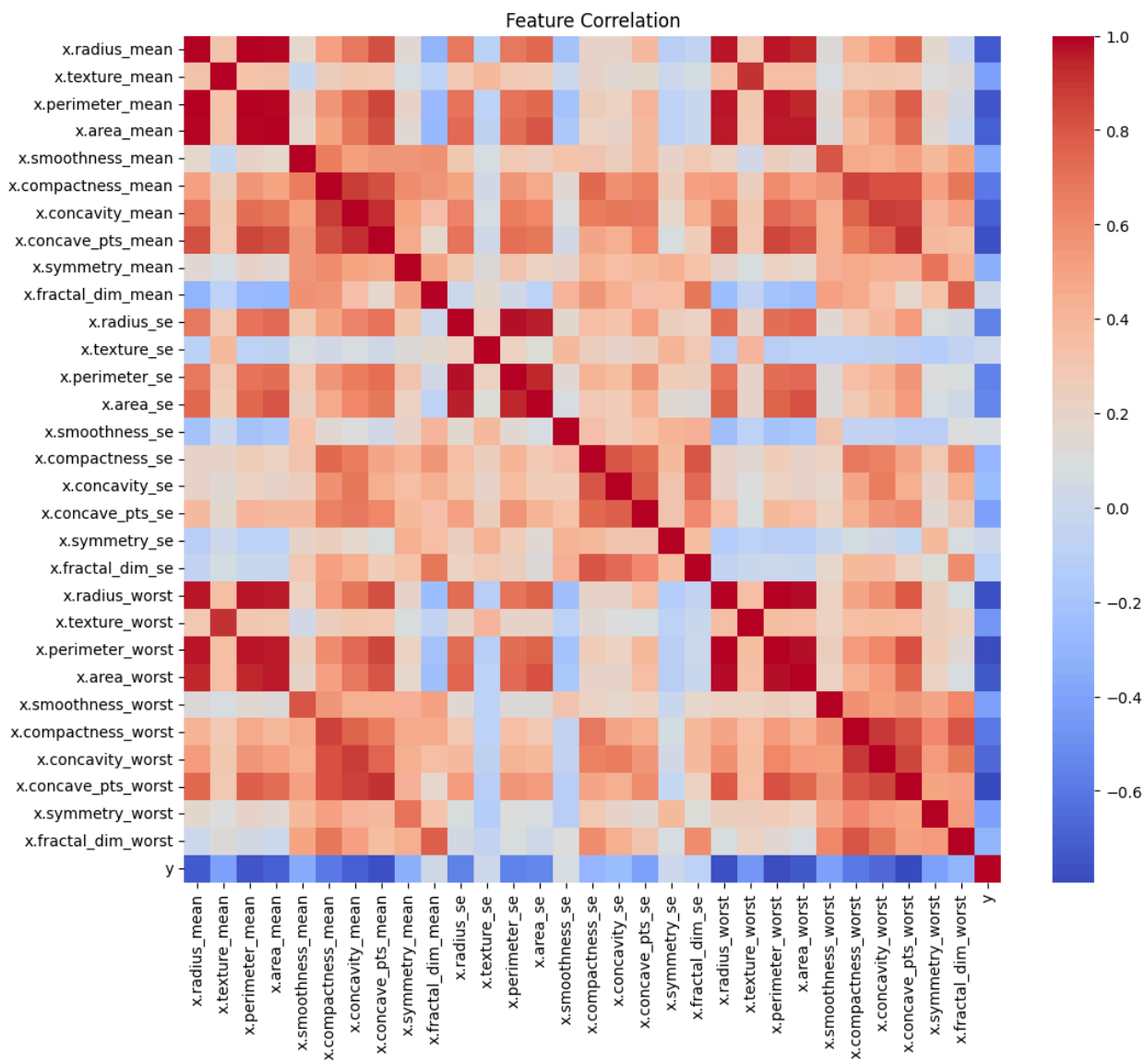
Out[7]:

	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean	x.smoothne
0	13.540	14.36	87.46	566.3	
1	13.080	15.71	85.63	520.0	
2	9.504	12.44	60.34	273.9	
3	13.030	18.42	82.61	523.8	
4	8.196	16.84	51.71	201.9	

5 rows × 31 columns

In [8]:

```
plt.figure(figsize=(12,10))
sns.heatmap(df.corr(), cmap="coolwarm")
plt.title("Feature Correlation")
plt.show()
```



In [9]:
```
X = df.drop(columns=['y'])
y = df['y']
```

In [10]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [11]:
```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [12]:
```
class Perceptron:

    def __init__(self, lr=0.01, epochs=100):
        self.lr = lr
        self.epochs = epochs

    def fit(self, X, y):

        self.weights = np.zeros(X.shape[1])
        self.bias = 0

        for _ in range(self.epochs):

            for i in range(len(X)):

                linear_output = np.dot(X[i], self.weights) + self.bias

                y_pred = 1 if linear_output >= 0 else -1

                if y[i] * y_pred <= 0:

                    self.weights += self.lr * y[i] * X[i]
                    self.bias += self.lr * y[i]

    def predict(self, X):

        linear_output = np.dot(X, self.weights) + self.bias
        return np.where(linear_output >= 0, 1, -1)
```

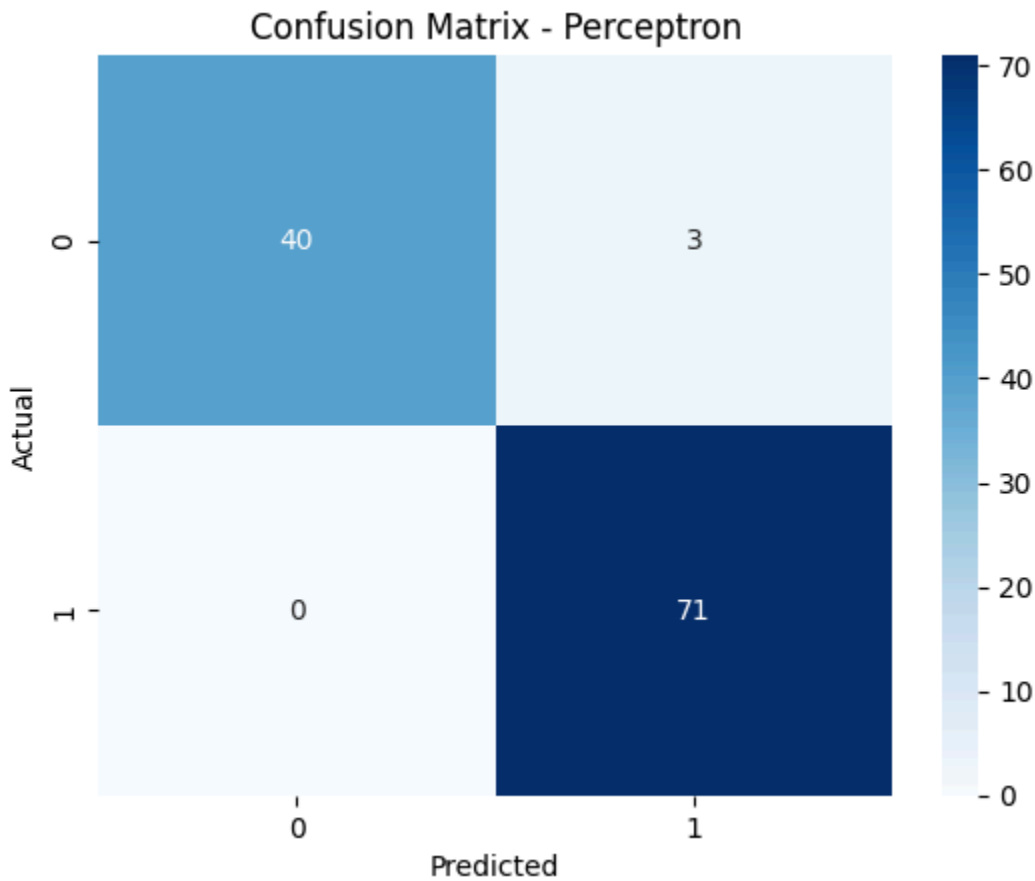
In [13]:
```
model = Perceptron(lr=0.01, epochs=100)

model.fit(X_train, y_train.values)
```

In [14]:
```
y_pred = model.predict(X_test)
```

In [15]:
```
accuracy = accuracy_score(y_test, y_pred)

print("Perceptron Accuracy:", accuracy)
```

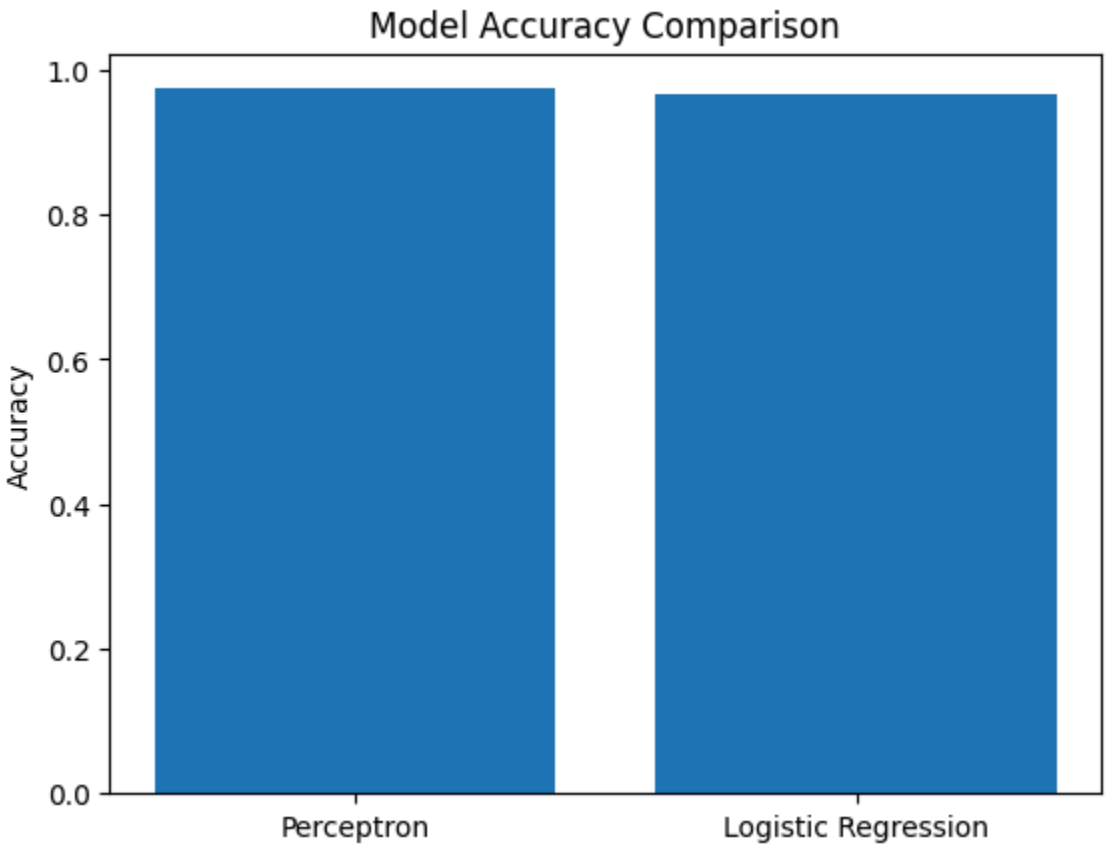
```
Perceptron Accuracy: 0.9736842105263158
```

In [16]:
```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")

plt.title("Confusion Matrix - Perceptron")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



In [17]:
```
log_model = LogisticRegression()

log_model.fit(X_train, y_train)

log_pred = log_model.predict(X_test)

log_accuracy = accuracy_score(y_test, log_pred)

print("Logistic Regression Accuracy:", log_accuracy)
```

```
Logistic Regression Accuracy: 0.9649122807017544
```

In [18]:
```
models = ["Perceptron", "Logistic Regression"]
scores = [accuracy, log_accuracy]

plt.bar(models, scores)
```

```
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")

plt.show()
```